

Atividade 1 — Mini E-commerce Distribuído

Relatório Técnico

Disciplina	Sistemas Distribuídos
Tecnologias	Java 17 · Spring Boot 3.2 · React 18 · Docker
Arquitetura	4 microsserviços · HTTP/REST · JWT · BCrypt

1. Como a comunicação entre os microsserviços foi implementada?

A comunicação entre todos os microsserviços foi implementada via **HTTP/REST** síncrono, utilizando o **RestTemplate** do Spring Framework para realizar chamadas entre serviços.

Fluxo de comunicação:

O **API Gateway** (porta 5000) atua como ponto de entrada único. Ele recebe todas as requisições do cliente e as encaminha para o microsserviço correto usando o `ProxyService`, que repassa o header *Authorization* (JWT) para os serviços internos.

O **Serviço de Pedidos** consulta o **Serviço de Produtos** via HTTP para validar existência e estoque antes de confirmar um pedido. A URL do serviço de produtos é injetada por variável de ambiente (`PRODUCTS_URL`), permitindo configuração sem recompilação.

Exemplo de chamada interna (`OrderService.java`):

```
Map product = restTemplate.getForObject(productsUrl + "/products/" +  
    productId, Map.class);
```

Cada microsserviço roda em porta separada (5001, 5002, 5012, 5003) e é iniciado de forma completamente independente. A comunicação nunca ocorre por chamadas de função direta — sempre por rede HTTP, garantindo o isolamento real entre serviços.

2. Qual estratégia de consistência foi adotada na replicação? Por quê?

Foi adotada **consistência forte (strong consistency)** com política **2-of-2 write**: toda operação de escrita precisa ser confirmada em ambas as réplicas antes de retornar sucesso ao cliente.

Justificativa:

Em um sistema de e-commerce, inconsistências no catálogo de produtos são críticas: um produto criado mas visível apenas em uma réplica poderia levar a situações em que pedidos são feitos para produtos inexistentes em parte do cluster. A consistência forte garante que qualquer leitura, em qualquer réplica, sempre reflita a escrita mais recente.

Funcionamento do ReplicationService:

1. Verifica se a réplica está acessível (GET /health). Se não estiver, rejeita a escrita.
2. Salva o produto localmente no arquivo JSON da instância primária.
3. Propaga via POST /internal/products/replicate para a réplica secundária.
4. Somente retorna 201 Created ao cliente após confirmação das duas réplicas.

Trade-off: maior latência em escritas (duas operações de rede) em troca de consistência absoluta. Para leituras, utilizamos round-robin entre as réplicas no Gateway, distribuindo a carga sem sacrificar consistência.

3. O que acontece com o sistema se o Serviço de Pedidos cair?

O sistema continua operando com **funcionalidade degradada**: os serviços de Usuários e Produtos permanecem totalmente funcionais. Apenas as operações de pedidos ficam indisponíveis.

Comportamento detalhado:

O **HeartbeatScheduler** do Gateway envia GET /health a cada 5 segundos para todos os serviços. Após 2 falhas consecutivas sem resposta do Serviço de Pedidos, o ServiceRegistry marca o serviço como *DOWN* e registra o evento no log com timestamp.

Qualquer requisição para /orders/* recebe resposta **503 Service Unavailable** com mensagem clara, sem que o cliente fique aguardando timeout. Os logs registram:

- Data e hora da falha detectada
- Número de tentativas falhadas
- Recuperação automática quando o serviço volta a responder

Quando o Serviço de Pedidos retorna, o próximo heartbeat bem-sucedido restaura o status para *UP* e registra a recuperação em log. Nenhuma intervenção manual é necessária.

4. Como o JWT garante que um usuário comum não consiga criar produtos?

O JWT carrega o campo **role** no payload, assinado com HMAC-SHA256 usando uma chave secreta compartilhada entre os serviços. Ninguém pode alterar o role sem invalidar a assinatura.

Fluxo de autorização:

1. No login, o UsersService gera um JWT com *userId*, *email*, *role* e *exp*.
2. O Gateway repassa o header *Authorization: Bearer <token>* para o ProductsService.
3. O ProductsService extrai e valida o JWT com `JwtService.validateToken()`.
4. Se o campo *role* não for "admin", retorna **403 Forbidden** antes de processar.

Trecho do ProductController.java:

```
Claims claims = jwtService.validateToken(token);
String role = (String) claims.get("role");
if (!"admin".equals(role)) {
    return ResponseEntity.status(403).body("Apenas administradores...");
}
```

Um usuário comum com role="user" recebe 403 mesmo que tente forjar o token, pois qualquer alteração no payload invalida a assinatura HMAC. A chave secreta nunca é exposta ao cliente.

5. Quais limitações a implementação possui em relação a um sistema real de produção?

Limitação	Impacto	Solução em Produção
Persistência em JSON	Sem transações ACID, risco de corrupção em escrita concorrente	PostgreSQL / MongoDB com transações
JWT sem revogação	Token roubado válido até expirar (24h)	Blacklist em Redis ou token rotation
Sem circuit breaker	Falhas em cascata se um serviço lento bloqueia threads	Resilience4j / Hystrix
Replicação síncrona simples	Sem quorum, sem leader election	Raft / Paxos ou Kafka
Gateway sem autoscaling	Ponto único de falha	Kubernetes + múltiplas réplicas do gateway
Sem TLS interno	Comunicação em texto claro entre serviços	mTLS com certificados internos
Heartbeat simplificado	Não detecta degradação parcial (lentidão)	Prometheus + métricas de latência

Apesar dessas limitações, a implementação cobre todos os conceitos fundamentais exigidos: decomposição em microsserviços independentes, replicação com estratégia de consistência definida, detecção de falha com heartbeat e autenticação/autorização via JWT com controle de roles.